The Developer EQ Sprint

A 2-week, 10-exercise program for engineers who can ship code but freeze in the meeting after.

By Kyle (ChaosKyle) — developereq.com

Howdy.

You can ship code. You freeze in the meeting after. Same boat I was in for ten years before I figured out the technical work was 30% of the job and the other 70% was the stuff nobody teaches you in a cert.

This sprint is the smallest possible loop that fixes that. **Two weeks. Ten standups. One sprint demo.** ~15 min a day. Let's mix it.

Sprint at a glance

```
Sprint 0   → Sprint Planning          (Sun before)
Week 1     → Mon-Fri (5 standups)     FOUNDATIONS
Week 1 Retro → Sat
Week 2     → Mon-Fri (5 standups)     PRESSURE & PRACTICE
Sprint Demo → Sat                     Show your work
```

Week 1 — Foundations:

1. The Skill Tree Audit
2. The 1:1 Reset
3. Standup Volume
4. Code Review as Communication

5. Async vs. Sync

Week 2 — Pressure & Practice: 6. The Calm Voice 7. The Disagreement Playbook 8. The Networking Algorithm 9. The Promotion Ask 10. The Follow-Up Superpower

How to use this workbook

- **Print it.** Write in it. Carry it.
- **Read it on screen.** Type your notes wherever (Notion, Obsidian, plain markdown).
- **Use AI?** There's a Claude Code skill (`/dev-eq`) and a generic AI prompt pack at github.com/ChaosKyle/developer-eq-sprint.

Whatever format. Just show up daily.

SPRINT 0: PLANNING

Read this before Day 1. Takes 10 minutes.

Why a sprint and not a "course"?

Because you've bought courses. We both have. They sit in a tab forever.

A sprint has:

- **A start and an end.** Two weeks. That's it. If you don't finish, the sprint failed and we move on.
- **A backlog.** 10 exercises. No more, no less. No "bonus content."
- **A definition of done.** You either did it or you didn't. No "I read it."
- **A demo.** At the end, you have something to show — even if it's only to yourself.

This is how your team ships features. It's how you ship a better version of yourself.

Sprint Goal

Pick the one that fits. Write it on Day 0.

- ☐ **Get promoted** in the next review cycle
- ☐ **Land a new role** at the level above my current one
- ☐ **Stop dreading** standups, 1:1s, or skip-levels
- ☐ **Get paid more** without changing jobs (raise / counter / scope)
- ☐ **Stop being the smartest person in the room** that nobody listens to
- ☐ Other: _____

Pre-Sprint Self-Audit

Rate yourself 1-5 (1 = "this is me at my worst", 5 = "I'm already good at this"):

Skill	1	2	3	4	5
Speaking up in standups when I have something to add	☐	☐	☐	☐	☐
Pushing back on a decision I disagree with	☐	☐	☐	☐	☐
Delivering critical feedback in a code review	☐	☐	☐	☐	☐
Asking for what I want (raise, scope, role)	☐	☐	☐	☐	☐
Following up with people I've met once	☐	☐	☐	☐	☐
Staying calm when production is on fire	☐	☐	☐	☐	☐
Driving my own 1:1 agenda	☐	☐	☐	☐	☐
Saying "I don't know" without it costing me credibility	☐	☐	☐	☐	☐

Total score: ___ / 40

Don't sweat the number. We'll redo this on Day 14 and the delta is what matters.

Sprint Cadence

```
Mon  Tue  Wed  Thu  Fri  Sat  Sun
─────────────────────────────────
[1]  [2]  [3]  [4]  [5]  RETRO ←  Week 1
[6]  [7]  [8]  [9]  [10] DEMO  ←  Week 2
```

One standup per workday. ~15 min. Retro on Saturday — no new exercise, just review what you wrote. Demo at the end.

If you miss a day, don't restart. Pick up tomorrow. The sprint accommodates real life.

WEEK 1: FOUNDATIONS

The week-1 stuff makes you good. Build the daily reps before you face the high-stakes moments in Week 2.

This week's backlog

Day	Exercise	Time
1	The Skill Tree Audit	15 min
2	The 1:1 Reset	15 min
3	Standup Volume	10 min + your standup
4	Code Review as Communication	20 min
5	Async vs. Sync	15 min
Sat	Week 1 Retro	15 min

Definition of done for the week: all 5 standups completed (or honestly skipped). Retro on Saturday.

DAY 1: The Skill Tree Audit

Burndown: 9 standups remaining

📦 Story Point

You've been leveling one skill tree your entire career — the technical one. AWS certs, language proficiency, system design, the works. That tree is maxed. The reason you're stuck is the *other* tree — the one nobody told you existed.

Today we map both. Not to feel bad. To see the gap.

✅ Definition of Done

Open a new doc (or use the space below). Make two columns:

Column A: Technical — every cert, language, framework, tool, deep skill you have.

Column B: Social/Career — every "soft" skill you'd put on a senior+ JD. Examples: mentoring, cross-team alignment, executive communication, conflict resolution, ambiguity tolerance, public speaking, written persuasion.

Star (★) the items in each column where you'd rate yourself 4 or 5.

🎯 Acceptance Criteria

- ⬜ Both columns have at least 8 items
- ⬜ You've starred your real strengths (no false modesty, no LinkedIn fluff)
- ⬜ You've identified the **2 biggest gaps** in column B that, if closed, would unblock your sprint goal

📝 Sprint Notes

```
Technical (Column A):
_____________________________________________
_____________________________________________
_____________________________________________

Social/Career (Column B):
_____________________________________________
_____________________________________________
_____________________________________________

My 2 biggest gaps:
1. _____________________________________________
2. _____________________________________________
```

⏭ Commit for Tomorrow

Tomorrow: your next 1:1. Look at your calendar. When is it? Block 5 minutes before it for prep.

```
Next 1:1 is on: __________  with: __________
```

> 🎙 ***REAL TALK***
>
> *My Column B was embarrassingly thin the first time I did this. Eight years of AWS, SRE, Splunk, Terraform — and under "social/career" I had: "good at writing." That was it. Not because I was bad at the others — I just hadn't named them. The audit isn't about feeling great. It's about seeing the gap so you can pick what to work on. The first time you write it down honestly is the start.*

DAY 2: The 1:1 Reset

Burndown: 8 standups remaining

📦 Story Point

Your 1:1 is the single most leveraged 30 minutes in your week and you're using it as a status update. Stop.

Status updates are what Slack is for. A 1:1 is the only time your manager is contractually obligated to talk about *you* — your career, your blockers, your next move. If you walk in with "here's what I shipped this week," you've thrown away the slot.

Best 1:1s I ever had were the ones where I drove. Worst were when I waited for my manager to ask questions and then mumbled answers.

✅ Definition of Done

Build a 1:1 agenda template you'll reuse every week. Three sections, in this order:

1. **Career / Growth** (10 min) — 1 question or topic about *you* getting better, getting promoted, getting unstuck. Always first. Always.
2. **Blockers / Asks** (10 min) — what do you need from your manager that only they can unblock? (Hint: "more context on the roadmap" counts. "Approval to use my Friday for X" counts.)
3. **Status / FYI** (10 min) — the stuff you would've led with. Now it's last. If you run out of time, it didn't matter.

🎯 Acceptance Criteria

- [ ] Template saved somewhere you'll find it next week (Notion, Obsidian, Apple Notes — pick one)
- [ ] Tomorrow's (or next) 1:1 has at least one career question pre-written
- [ ] You've identified one "ask" you've been sitting on

📝 Sprint Notes

```
My 1:1 template lives at: _________________________

The career question I'll bring next 1:1:
_____________________________________________________
_____________________________________________________

The ask I've been sitting on:
_____________________________________________________
```

⏭ Commit for Tomorrow

Tomorrow's exercise happens *during* a real meeting. Look at your calendar — pick the standup or team meeting where you'll practice.

```
Tomorrow's lab meeting: __________  at __________
```

DAY 3: Standup Volume

Burndown: 7 standups remaining

📦 Story Point

There are two kinds of broken standups: the engineer who never speaks (invisible) and the engineer who narrates every commit (annoying). You're probably one. Maybe both, depending on the day.

The fix isn't "speak more" or "speak less." It's *speak with intent*.

Think about your audio mix. The lead vocal isn't the loudest thing the whole song — it ducks during the chorus, comes back for the verse. Your standup voice should ride the same fader. Loud when you're driving the work, quiet when you're not.

✅ Definition of Done

In your next standup (or team sync), do exactly ONE of these:

Option A (if you're the quiet one): Share a blocker — even a small one. Format: "I'm stuck on X. I think the path forward is Y. Anyone done this before / have time to pair for 20 min?"

Option B (if you're the loud one): Stay quiet for the first 3 updates. Don't interject. Don't add color. Just listen. Then when it's your turn, say only what's *changed* since yesterday — no preamble.

🎯 Acceptance Criteria

- [ ] You did the option you picked (no chickening out)
- [ ] You noticed one thing about how *other people* update the team that you can steal
- [ ] You didn't apologize for taking up time (kill phrases like "sorry, just wanted to add...")

📝 Sprint Notes

```
Which option I picked: A / B

What I said:
_____________________________________________

What I stole from someone else's update:
_____________________________________________

Reaction (mine or anyone else's):
_____________________________________________
```

⏭ Commit for Tomorrow

Tomorrow: pull up your last 5 code review comments. We're rewriting one.

```
Where I review code: GitHub / GitLab / Bitbucket / other ____
```

DAY 4: Code Review as Communication

Burndown: 6 standups remaining

📦 Story Point

Your code review comments are seen by more people than your standup voice. They're permanent. They get screenshotted. They show up in promotion packets — yours and theirs.

And most engineers' comments read like a robot wrote them on a bad day.

Two failure modes:

1. **Too harsh:** "This is wrong." → reader hears "you are dumb." → defensive PR ping-pong → wasted hours
2. **Too soft:** "Have we considered maybe possibly a different approach perhaps?" → no actionable feedback → ship the bug → blame falls on the reviewer for not catching it

The win is *direct + warm*. Pick the change you want. Explain why in one sentence. Move on.

✅ Definition of Done

Pull up your last 5 code review comments (or the last comment you almost sent). Pick the one you'd be most embarrassed to have read aloud at all-hands.

Rewrite it three ways:

1. **The blunt version** — direct, no fluff, here's the change I want
2. **The warm version** — same change, but with one sentence of "why" and one sentence acknowledging the author's intent
3. **The teaching version** — for a more junior reviewer, where you're trying to grow them

Pick which one fits the actual situation and use that next time.

🎯 Acceptance Criteria

- [ ] All three versions written (don't skip the blunt one — that's where you'll learn)
- [ ] The warm version is shorter than 4 sentences
- [ ] You've identified which version your team's culture rewards

📝 Sprint Notes

```
Original comment:
--------------------------------------------------

Blunt version:
--------------------------------------------------

Warm version:
--------------------------------------------------

Teaching version:
--------------------------------------------------

Which one I'll default to going forward, and why:
--------------------------------------------------
```

⏭ Commit for Tomorrow

Tomorrow: think about the last week of your Slack. We're picking one thread that should've been a meeting (or one meeting that should've been a thread).

> 🎙 ***REAL TALK***
>
> *A comment I left on a junior dev's PR years ago: "Why are you nesting these like this? This is going to be unreadable in 6 months." No suggestion. No why. No acknowledgment they were trying to solve a real problem. Took me 4 years to figure out why nobody on that team would pair with me. Don't be 27-year-old me.*

DAY 5: Async vs. Sync

Burndown: 5 standups remaining

📦 Story Point

Half the meetings on your calendar shouldn't exist. Half the Slack threads on your screen should've been a 15-minute call.

Engineers default to whichever channel matches their personality — quiet folks default to async, loud folks default to sync. Both are wrong half the time.

Quick rule that's saved me hours: **if there's disagreement or ambiguity, go sync. If it's a status update, go async. If it's a decision involving more than 3 people, neither — write a doc and ask for comments.**

✅ Definition of Done

Audit the last 7 days of your work communication. Find:

1. **One Slack thread** that went 20+ messages and would've been resolved in a 15-min call
2. **One meeting** you sat in that should've been a Slack message or doc

For each, write the *one sentence* you would say next time to redirect.

Examples:

- "This is getting tangled — want to hop on a call for 15?"
- "Quick async question, no need for a meeting — [question]?"
- "Let me write this up as a doc and tag you for comments instead of meeting."

🎯 Acceptance Criteria

- [ ] You found a real example of each (not hypothetical)
- [ ] Your redirect sentences are < 20 words each
- [ ] You used one of them this week

📝 Sprint Notes

```
Thread that should've been sync:
_____________________________________________
My redirect sentence: ____________________________

Meeting that should've been async:
_____________________________________________
My redirect sentence: ____________________________
```

⏭ Commit for Tomorrow

Tomorrow is the **Week 1 Retro**. No new exercise — just 15 minutes reviewing what you wrote.

WEEK 1 RETRO

Saturday. 15 minutes. No new exercise.

Open Days 1-5. Read what you wrote. Don't judge it — just look.

Then answer:

What worked?

> _____
> _____

What didn't?

> _____
> _____

What surprised me?

> _____
> _____

What's one thing I'll carry into Week 2?

> _____

If you missed a day or two, no shame — almost everyone does. Pick up Monday.

WEEK 2: PRESSURE & PRACTICE

Week 1 was foundations — the stuff you do every day. Week 2 is the high-stakes stuff. Incidents, disagreements, asking for what you're worth.

These are the moments that change the trajectory of your career. The week-1 stuff makes you good. Week-2 stuff makes you visible.

This week's backlog

Day	Exercise	Time
6	The Calm Voice (incident leadership)	20 min
7	The Disagreement Playbook	15 min
8	The Networking Algorithm	30 min (most is sending)
9	The Promotion Ask	30-45 min ⭐
10	The Follow-Up Superpower	20 min
Sat	Sprint Demo	30 min

Day 9 is the highest-leverage exercise in the sprint — block real time for it. The Sprint Demo on Saturday is the most important page in this workbook.

DAY 6: The Calm Voice

Burndown: 4 standups remaining

📦 Story Point

Production is on fire. Channel is filling up with "is anyone looking at this?" and "I'm seeing 500s." Everyone's in panic mode.

The person who runs the incident isn't the most senior. It's the one whose voice doesn't shake.

Calm in an incident is a learnable skill, not a personality trait. I learned it on-call at AWS by being the loudest panicker first, watching the senior SREs work, and stealing their playbook.

The playbook is short:

1. **Acknowledge:** "I'm on it." (Now everyone knows someone's driving.)
2. **Assess:** "Pulling logs / checking metrics. Back in 5." (Buys you time.)
3. **Communicate:** Update every 5-10 min, even if the update is "still investigating." Silence is what causes panic.
4. **Decide & document:** State the action you're taking and why, in the channel, in writing. Not "I think we should probably maybe roll back" — "Rolling back deploy abc123 because error rate spiked at 14:32. Will confirm impact in 5."

✅ Definition of Done

Write your **Incident Response Card.** A one-page doc with:

- Your top 3 dashboards/links (paste the URLs)
- Your rollback command (or how to find it)
- The escalation path (who to page after how long)
- The 4 phrases above, copy-paste ready, in your voice

Print it or pin it. It's there for the next 2am page.

🎯 Acceptance Criteria

- [ ] All 4 sections filled in with real values, not placeholders
- [ ] The doc is somewhere you can reach in 30 seconds at 2am
- [ ] You've shared it with one teammate (bonus: ask them what's on theirs)

📝 Sprint Notes

```
My incident response card lives at: _____________

Dashboards: ____________________________________
Rollback: ______________________________________
Escalation: ____________________________________

Phrases I'll use:
1. _____________________________________________
2. _____________________________________________
3. _____________________________________________
4. _____________________________________________
```

⏭ Commit for Tomorrow

Tomorrow: think of a decision you've disagreed with at work but stayed quiet about.

📋 *EXAMPLE — what a filled-in card looks like*

```
INCIDENT RESPONSE CARD — Kyle

Dashboards (open these first):
1. CloudWatch — prod alarms — [link]
2. Datadog — service overview — [link]
3. Honeycomb — error tracing — [link]

Rollback (deploy):
  ./scripts/rollback.sh <service> <commit-sha>
  (commit SHAs in #deploys channel)

Escalation:
  - 5 min: post in #incidents
  - 10 min: page on-call EM
  - 15 min: page director

Phrases (paste into channel verbatim):
1. "I'm on it. Pulling logs now."
2. "Update in 5 — checking [X] before I act."
3. "Still investigating. Nothing actioned yet."
4. "Action: rolled back deploy <sha>. Confirming impact in 5."
```

Don't overthink it. Yours doesn't need to be pretty. It needs to be reachable in 30 seconds at 2am.

DAY 7: The Disagreement Playbook

Burndown: 3 standups remaining

📦 Story Point

You disagree with the architecture decision. Or the deadline. Or the hire. And you said nothing.

Six months later, the thing you predicted happens. You feel vindicated and unheard. You start drafting your resignation in your head.

There's a path between "shut up and execute" and "burn the bridge." It's called *committing the disagreement to writing, before the decision ships*.

The format that's never failed me:

> *"I want to raise a concern before we move forward. I might be wrong, and I'll commit to the decision either way. Here's what I'm worried about: [1 sentence]. Here's what I'd want to see to feel better: [1 sentence]. Here's the alternative I'd consider: [1 sentence]."*

Three sentences. Written down. Sent to the decision-maker.

If they hear you and adjust — great. If they hear you and say "noted, we're moving forward" — you've still done your job. And six months later when the thing happens, there's a record. Not for "I told you so" — for *trust*. People remember who flagged it.

✅ Definition of Done

Pick a real decision happening at work in the next 2 weeks (a roadmap call, a tech choice, a priority, a hire). One you have a real opinion on.

Write the 3-sentence disagreement, even if you don't end up sending it. Even drafting it changes what you'll say in the next meeting.

🎯 Acceptance Criteria

- [ ] The decision is real, not hypothetical
- [ ] Your draft is exactly 3 sentences (don't pad it)
- [ ] You've committed: send it / say it in the next meeting / let it ride (and you know which and why)

📝 Sprint Notes

```
The decision:
_____________________________________________

My 3-sentence disagreement:
1. _____________________________________________
2. _____________________________________________
3. _____________________________________________

What I'll do with it: send / say / hold
Why: _____________________________________________
```

⏭ Commit for Tomorrow

Tomorrow: pull up your LinkedIn or contact list. We're picking 3 people you haven't talked to in 6+ months.

DAY 8: The Networking Algorithm

Burndown: 2 standups remaining

📦 Story Point

"Networking" sounds gross because most engineers picture conferences and LinkedIn requests from strangers. That's not networking. That's lottery tickets.

Real networking is *maintaining weak ties*. The ex-coworker, the person you met at one meetup, the friend-of-a-friend on a podcast you liked. Studies on job mobility (Granovetter, 1973 — the OG network science paper) show that **most career-changing opportunities come through weak ties, not close friends.** Your close friends know what you know. Weak ties know what you don't.

The algorithm: **once a quarter, message 3 weak ties. No ask. Just a check-in.**

That's it. No "hey would you mind if I picked your brain." No coffee chat ambush. Just "hey saw you posted about X, congrats / sympathies, I'm doing Y these days, hope you're well."

You do this for a year and you have a 12-person network of people who actually remember you.

✅ Definition of Done

Send 3 messages today. Right now. Not "I'll do it later." Today.

Format:

- Pick someone specific (LinkedIn, old phone contact, ex-coworker)
- 3-5 sentences
- One thing about *them* (something they posted, something they shipped, kid/pet update they shared)
- One thing about *you* in one sentence
- No ask

🎯 Acceptance Criteria

- [ ] All 3 messages sent before you go to bed
- [ ] None of them are templated — each is specific to that person
- [ ] You've added a recurring calendar reminder for next quarter

📝 Sprint Notes

```
Person 1: _____  Sent: Y/N  Notes: _____
Person 2: _____  Sent: Y/N  Notes: _____
Person 3: _____  Sent: Y/N  Notes: _____

Recurring reminder set for: ____________________
```

⏭ Commit for Tomorrow

Tomorrow's the big one. We're drafting your case for the next level — even if you're not asking for it yet.

DAY 9: The Promotion Ask

Burndown: 1 standup remaining

📦 Story Point

You're not getting promoted because you "deserve" it. Lots of people deserve it. You're getting promoted because someone in a calibration meeting can articulate, in one paragraph, why you're already operating at the next level.

That someone is supposed to be your manager. But your manager has 8 reports, three competing initiatives, and is forgetting half the stuff you did 6 months ago.

So you write the paragraph for them.

This isn't politicking. It's reducing the cognitive load on the person who already wants to advocate for you. Make it easy.

✅ Definition of Done

Write your **Promotion Brief.** One page. Three sections:

1. **What level am I asking for, and why am I already operating there?** (3 sentences max)
2. **3 specific projects/wins from the last 6 months that demonstrate it** (1 paragraph each, with metrics if you have them — money saved, hours reduced, scope owned, people grown)
3. **What's the one thing I'm working on that would close any remaining gap?** (1 sentence — bonus: tie it to the sprint you just did)

Send it to your manager *before* your next 1:1, not as a surprise drop. Frame: "Here's a brief I drafted on what I'm working toward — would love to talk through it next 1:1."

If you're not in a place to send it: still write it. The act of writing it changes how you talk about your work.

🎯 Acceptance Criteria

- [ ] One page, three sections, all complete
- [ ] Each project has at least one quantified outcome (numbers > adjectives)
- [ ] You've decided: send to manager now / save for review cycle / use as personal brag doc

📝 Sprint Notes

```
Level I'm aiming for: ____________________________

Why I'm already operating there:
_____________________________________________
_____________________________________________

Project 1: ______________________________________
Project 2: ______________________________________
Project 3: ______________________________________

Closing-gap commitment:
_____________________________________________

What I'll do with this brief: _____________________
```

⏭ Commit for Tomorrow

Tomorrow: the last standup. Open your DMs and inbox. You owe somebody a follow-up.

📋 *EXAMPLE — what a real promotion brief looks like (anonymized)*

> *PROMOTION BRIEF — [Name], Senior → Staff Engineer*
>
> *Why I'm already operating at Staff:*
> *Over the last 6 months I've owned cross-team initiatives, mentored two senior engineers, and reduced our org's biggest reliability risk without being asked. The remaining gap is visibility, not ability.*
>
> *1) Reduced production incident MTTR by 41% (38min → 22min)*
> *Owned the on-call playbook rewrite, ran 3 game days, retrained 6 engineers. Quantified via PagerDuty data Q3 vs Q4.*
>
> *2) Migrated checkout service off the legacy auth layer*
> *Cross-team initiative across Platform + Payments. 4 month effort. Unblocked the SSO project (now shipping in Q1) and removed our only remaining single-tenant DB. Estimated $180K/yr infra savings.*
>
> *3) Mentored two senior engineers through their own promo cycles*
> *Both got promoted; both cited 1:1s with me as a key factor in their packets. (Manager confirms.)*
>
> *Closing-gap commitment:*
> *Running the architecture review for the new fraud detection service next quarter. This puts me in the room with leadership weekly and gives me a written artifact to point to in the next calibration.*

Notice: every project has a number. Every claim is verifiable. The "why I'm already operating there" is 3 sentences, not 3 paragraphs. Steal the format.

DAY 10: The Follow-Up Superpower

Burndown: 0 standups remaining

📦 Story Point

Every senior person I respect has a stack of "I owe X a reply" sitting in their head. Most never send.

Following up — *actually* following up — is the rarest social skill among engineers. Recruiters who follow up book more candidates. PMs who follow up ship more features. Engineers who follow up after a hallway conversation get the next opportunity.

It's not magic. It's just doing the thing nobody does.

✅ Definition of Done

Spend 20 minutes today doing follow-ups you've been sitting on. Pick from:

- The recruiter who reached out 3 months ago and you ghosted
- The conference contact whose card is still in your laptop bag
- The teammate who helped you debug something — did you ever say thanks?
- The intro a friend made that you never followed through on
- The "let's grab coffee" thread that died in October

Send at least 3.

Format is the same as Day 8: short, specific, low-ask. "Hey, you helped me with X back in [month] — wanted to circle back and let you know I shipped it / it worked / I owe you one."

🎯 Acceptance Criteria

- [ ] At least 3 follow-ups sent
- [ ] Each one references something specific (not "just checking in")
- [ ] You've added a "follow-up Friday" — a recurring 15-min weekly slot to keep this going

📝 Sprint Notes

```
Follow-up 1: _____________________________________
Follow-up 2: _____________________________________
Follow-up 3: _____________________________________

Follow-up Friday slot: ____________________________
```

⏭ Commit for Tomorrow

Tomorrow is your **Sprint Demo.** No new exercise — you're presenting your work to yourself.

SPRINT DEMO

Day 11. 30 minutes. The most important page in this workbook.

Pull up Day 0 (your self-audit) and Days 1-10 side by side.

Re-do the self-audit

Same scale, same questions. No peeking at your old scores.

Skill	1	2	3	4	5
Speaking up in standups when I have something to add	☐	☐	☐	☐	☐
Pushing back on a decision I disagree with	☐	☐	☐	☐	☐
Delivering critical feedback in a code review	☐	☐	☐	☐	☐
Asking for what I want (raise, scope, role)	☐	☐	☐	☐	☐
Following up with people I've met once	☐	☐	☐	☐	☐
Staying calm when production is on fire	☐	☐	☐	☐	☐
Driving my own 1:1 agenda	☐	☐	☐	☐	☐
Saying "I don't know" without it costing me credibility	☐	☐	☐	☐	☐

Total score: ____ / 40

Day 0 score: ____ / 40

Delta: ____

Sprint Velocity

Number of exercises actually completed: ___ / 10

(If it's less than 7, that's fine — the next sprint, you start with what didn't happen.)

What shipped?

The 3 things you actually *did* during this sprint that you wouldn't have done otherwise:

1.

2.

3.

What's the next sprint?

You're not done. You're just done with *this* sprint.

Pick ONE thing to focus on for the next two weeks:

- [ ] Re-run any 3 standups that didn't land the first time
- [ ] Send the promotion brief and have the conversation
- [ ] Apply Day 6 (calm voice) to a real incident or high-stakes meeting
- [ ] Commit to Follow-up Friday for a full quarter
- [ ] Start the book — go deeper on one of the chapters that hit (developereq.com)
- [ ] Other: _____

SPRINT RETROSPECTIVE — WHAT'S NEXT

You did the sprint. That puts you in the top 5% of people who downloaded this thing — most never start. So first: nice work.

If something here clicked, here's where to go deeper:

The book — $20

Developer EQ is the full source. 16 chapters. Every concept in this sprint is one chapter, expanded with stories, frameworks, and the actual playbooks I used to go from "good engineer who got stuck" to "engineer who gets promoted."

→ **developereq.com**

The cohort — $500

Developer EQ Live Cohort. 4 weeks. Tuesday nights, 7pm CST. Max 20 people. Lifetime Discord access.

You do the sprint exercises in a room of other engineers, with me running the room. It's the difference between practicing scales alone in your bedroom and playing in a band.

Three cohorts in 2026: **May 12, June 9, July 7.**

→ **developereq.com/cohort**

At DevOps Days? Use code `DEVOPSDAYS` for $100 off.

Connect

I'm @chaoskyle on most platforms. New blog posts go up at developereq.com/blog. Reach out — I read everything (and follow up).

Built by Kyle (ChaosKyle) — kyle@ksbdigital.com — developereq.com

If you used this and shipped something, I'd love to hear it. Tag me. Send me an email. The follow-up superpower works both ways.